# Home Assignment Week-02- Day-01

**Task 1:** Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

**Chat Model:** ChatGpt
**Domain :** E-commerce

**Context**:
Act as a **Senior QA/Test Architect with 10+ years of experience in E-commerce applications** (such as marketplaces, retail platforms, order management systems, payment integrations, and microservices-based architectures). You have deep expertise in:

- End-to-end **test strategy and planning**
- **Risk-based testing methodologies**
- **Defect management and metrics** (defect density, leakage, DRE)
- **Automation strategy** (UI, API, performance, and integrations)
- Testing **high-traffic, customer-facing applications**
- Handling **Agile/DevOps environments with CI/CD pipelines**
- Ensuring quality across **web, mobile, and backend systems**

You are responsible for creating a **comprehensive, real-world Test Strategy Document** aligned with industry standards (ISTQB, IEEE 829/29119) and modern E-commerce QA practices.

**Instructions:**

**1. Interactive Requirement Gathering**

- Ask exactly **5 structured questions**, one at a time.
- Each question should target a **critical test strategy input area**, such as:

    - Application scope (catalog, cart, checkout, payments, etc.)
    - Integrations (payment gateways, shipping, third-party services)
    - User load / traffic expectations
    - Environments & deployment model
    - Business-critical workflows
- Wait for the user's response before proceeding to the next question.

---

**2. Test Strategy Creation**

Based on the collected inputs, generate a **complete and realistic Test Strategy Document**, including:

- Scope & objectives
- Test approach:
    - Functional testing (UI, API)
    - Integration testing (payments, logistics, inventory)
    - Regression testing
    - UAT support
- **Risk-based testing strategy**
    - Risk identification
    - Impact vs probability matrix
    - Prioritization of critical flows (e.g., checkout, payment)
- **Defect management approach**
- **Defect metrics**:
    - Defect Density
    - Defect Leakage
    - Defect Removal Efficiency (DRE)
    - Expected Defect Ratio benchmarks for E-commerce
- Test environment strategy
- Test data management (users, orders, payment scenarios)

- **Automation strategy**:
  - UI automation (web/mobile)
  - API automation
  - CI/CD integration
  - Designed from a **Senior Automation Engineer (10+ years experience)** perspective
- Performance & scalability testing considerations (peak sales, flash sales)
- Security testing considerations (payment, user data)
- Entry & Exit criteria
- Deliverables & timelines
- Roles & responsibilities

---

**3. Output Requirements**

Generate:

- 📄 **Test Strategy Document in Word format (.docx)**
  - Enterprise-ready, clean structure, client presentation quality
- 📊 **Review Comments Tracker in Excel (.xlsx)** including:
  - Reviewer Name
  - Comments
  - Severity
  - Status
  - Owner
  - Resolution

---

**4. Quality Expectations**

The output must:

- Be **realistic and implementation-ready**

- Reflect **actual E-commerce challenges** (high traffic, payment failures, cart abandonment, etc.)
- Include **practical insights and assumptions**
- Be written in a **professional senior QA tone**
- Avoid generic content — make it **domain-specific and actionable**

---

✅ **Optional Enhancement**

Include **assumptions wherever inputs are missing**, clearly marked.

**Ecommerce Test Strategy Document attached**

Ecommerce_Test_Strategy.docx

Review_Tracker.xlsx

**Task 2 :** Design the automation framework structure using a chain-of-thought approach

**Chat Model Used** : Claude AI

As an expert Automation Tester, generate a Selenium automation framework structure using Java to automation opencart ecommerce browser based applicaton.

- Ask me 5 questions one by one to guide the framework creation, only moving to the next question after I provide an answer.
- The framework must follow industry-standard practices.
- [STRICT]Once the framework is generated, review it strictly as a Test Automation Lead, identify and fix any errors, and provide the corrected framework code compressed into a .zip file.

- [CRITICAL] After framework creation, validate integration with CI/CD pipelines, fix any build or deployment issues, and provide the pipeline-enabled framework packaged in a .zip file.
- [STRICT REVIEW] Once the framework is delivered, conduct a peer-level audit for performance bottlenecks, optimize the code, and provide the corrected optimized framework in .zip format.
- [FINAL VALIDATION] After framework generation, map requirements to test cases in a traceability matrix, ensure coverage, fix gaps, and provide the corrected framework with RTM included in a .zip file.

Opencart.zip